DATA STRUCTURES PROJECT REPORT

Student Marks Management System

A dynamic singly linked list implementation in C

SUBMITTED BY

Darshit Tank
Roll No. 1011

Arpan Savaliya
Roll No. 1006

Dev Shah
Roll No. 1014

Abstract

Managing student academic records is one of the most common tasks in schools and colleges. Traditional programs rely on fixed-size arrays, which either waste memory or run out of capacity. This project builds a console-based **Student Marks Management System** in C using a dynamic **singly linked list**. Memory is requested for each student with `malloc()` and returned with `free()`, so nothing is wasted and there is no fixed limit on class size.

1 Add
Insert new student records dynamically.

2 Display
List every enrolled student.

3 Search
Find a student by unique roll number.

4 Delete
Remove a record and free its memory at once.

5 Statistics
Total students, average marks and class topper.

6 Exit
Leave the program cleanly.

Objectives

- Replace fixed arrays with dynamic memory so no space is wasted.
- Support the six core operations through one simple menu.
- Free memory immediately on deletion to avoid leaks.
- Show pointer manipulation and time complexity in a real, small program.

1 Introduction

The system lets a teacher, administrator or student track academic performance in an organised way. A beginner version usually stores the records in a fixed array:

```
struct Student students[100];   // fixed array for 100 students
```

Wasted memory
With 5 students, space for the other 95 is reserved and unused.

Fixed limit
The 101st student cannot be added because the array is full.

Costly deletion
Removing student #2 shifts every later student one place forward.

The solution: a linked list allocates memory on demand. Each new student gets exactly one node from `malloc()`, and a deleted student's node goes straight back to the operating system through `free()`.

2 Data structure: singly linked list

In a singly linked list, data lives in separate blocks called **nodes**. Each node holds the student's data and the memory address of the next node, so the nodes form a one-way chain that can be followed from the start to the end.

Node
One student record: roll number, name and marks, plus a link to the next node.

head pointer
Points to the very first student. An empty list has head equal to NULL.

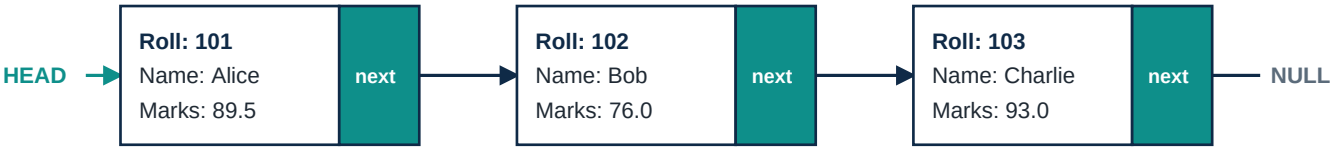
next pointer
Stores the address of the following node, so the chain can be walked.

NULL
The last node points to NULL, which signals the end of the list.

C structure definition

```
struct Student {
    int roll;              // unique identifier for the student
    char name[30];         // student's full name
    float marks;           // marks scored (0.0 to 100.0)
    struct Student* next;  // pointer to the next node
};
```

How the nodes are connected



Memory used by one node

Field	Type	Size	Purpose
roll	int	4 bytes	Unique roll number
name	char[30]	30 bytes	Student's full name
marks	float	4 bytes	Marks scored
next	struct Student*	8 bytes (64-bit)	Address of the next node
Total		46 bytes	About 48 bytes after compiler padding

Array vs linked list

Feature	Fixed array	Singly linked list (used here)
Memory allocation	Fixed at compile time	Allocated at runtime as needed
Size limit	Set by array size (e.g. 100)	None, apart from available RAM
Memory use	Wasted when partly filled	Only for existing records
Insertion at end	O(1) until the array is full	O(N): walk to the end, then attach
Deletion	O(N): find, then shift later elements	O(N) to find, then O(1) to unlink and free(); no shifting

3 The six functions

Each function handles exactly one user requirement.

3.1 `addStudent()` adds a student

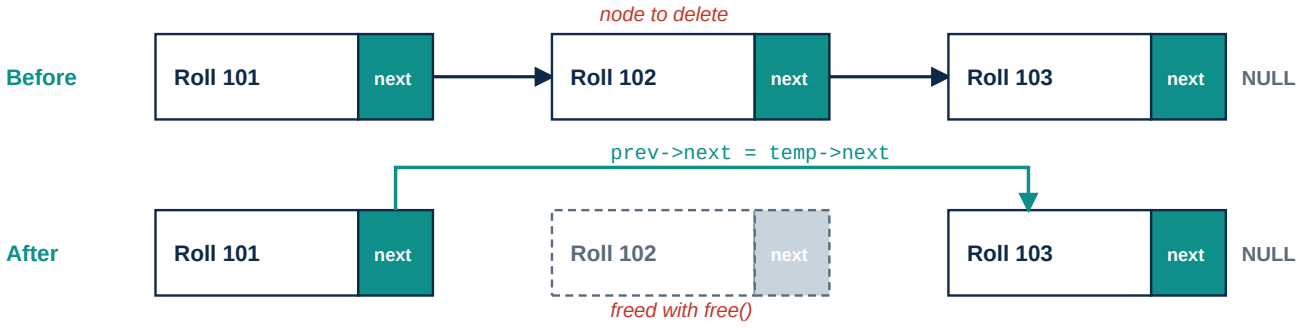
A new node is created with `malloc()`, filled from user input and given `next = NULL` because it will be the last node. If the list is empty it becomes the head; otherwise a pointer `temp` walks to the last node and links the new one.

```
void addStudent() {
    struct Student* newNode = (struct Student*)malloc(sizeof(struct Student));
    printf("Enter Roll Number: ");  scanf("%d", &newNode->roll);
    printf("Enter Name: ");         scanf(" %29[^\n]", newNode->name);
    printf("Enter Marks: ");        scanf("%f", &newNode->marks);
    newNode->next = NULL;
    if (head == NULL) { head = newNode; return; }
    struct Student* temp = head;
    while (temp->next != NULL) temp = temp->next;   // reach the last node
    temp->next = newNode;
}
```

3.2 `deleteStudent()` removes a student and frees memory

The function keeps two pointers, `temp` (current node) and `prev` (the node before it). When the roll number matches, the previous node is made to skip the target, and the target's memory is released. Deleting the first node simply moves `head` forward.

```
void deleteStudent() {
    int roll;  printf("Enter Roll Number to delete: ");  scanf("%d", &roll);
    struct Student *temp = head, *prev = NULL;
    while (temp != NULL && temp->roll != roll) { prev = temp; temp = temp->next; }
    if (temp == NULL) { printf("Student not found!\n"); return; }
    if (prev == NULL) head = temp->next;    // first node: move head forward
    else prev->next = temp->next;           // otherwise bypass the node
    free(temp);                             // return memory to the OS
}
```



3.3 `displayStudents()`

Prints "No student records found!" if `head == NULL`. Otherwise loops `while (temp != NULL)`, printing each record and stepping with `temp = temp->next`.

3.4 `searchStudent()`

Reads a roll number and compares it with every node from the head. On a match it prints name and marks; reaching `NULL` prints "Student not found".

3.5 `classStatistics()`

One traversal keeps `count++` and `sum += marks` and tracks the highest mark with its owner. Prints total students, average (`sum / count`) and the topper.

3.6 `main()`

A `while (1)` loop shows menu options 1 to 6; a `switch` calls the matching function; option 6 ends the program with `exit(0)`.

4 Complexity analysis

Every operation is O(N) in the worst case because the list can only be reached by walking from `head`.

Operation	Best	Average	Worst	Why
Add student	O(1) if empty	O(N)	O(N)	Walk to the last of N nodes
Display all	O(N)	O(N)	O(N)	Visits every node once
Search student	O(1) at head	O(N)	O(N)	Linear search along pointers
Delete student	O(1) at head	O(N)	O(N)	Finding is O(N); unlink and free() are O(1)
Class statistics	O(N)	O(N)	O(N)	Visits every node for sum and maximum

Space: auxiliary space is O(1) because no extra arrays or recursion are needed. Total memory is O(N) for N students, about 48 bytes per student.

5 Sample program run

```
========================================
    STUDENT MARKS MANAGEMENT SYSTEM
========================================
1. Add Student        4. Delete Student
2. Display Students   5. Class Statistics
3. Search Student     6. Exit
----------------------------------------
Enter your choice (1-6): 1
Enter Roll Number: 101   Enter Name: Alex   Enter Marks: 88.5
--> Student added successfully!
Enter your choice (1-6): 1
Enter Roll Number: 102   Enter Name: Bob    Enter Marks: 94.0
--> Student added successfully!
Enter your choice (1-6): 2
--- Student Records ---
Roll No: 101 | Name: Alex | Marks: 88.50
Roll No: 102 | Name: Bob  | Marks: 94.00
Enter your choice (1-6): 5
--- Class Statistics ---
Total Students : 2
Average Marks  : 91.25
Highest Score  : 94.00 by Bob
Enter your choice (1-6): 6
Exiting program. Goodbye!
```

6 Conclusion and future enhancements

The system shows the core strengths of a singly linked list. By using dynamic allocation with `malloc()` and pointer manipulation, it wastes no memory, has no artificial limit on class size, and keeps the C code modular and easy to study and present.

File handling

Save nodes to a .txt or .csv file so records persist after the program closes.

Sorting

Apply bubble sort or merge sort to rank students by marks or order them by roll number.

Doubly linked list

Add a prev pointer so records can be traversed backwards.